

Documentation projet

Widad

March 12, 2026

Abstract

Documentation du travail effectué. Dernier edit: 12/03/2026

1 `evaluer_reponses.py`

AUTEUR: Widad

C'est le script principal qui évalue les tickets. Il contient aussi une fonction de comparaison des différents modèles.

A l'exécution du fichier 2 choses se produisent:

- Il génère le fichier `tickets_evalues_scores.json`.
- Il affiche également dans le terminal une comparaison des différents modèles en se basant sur les tickets correspondants.

2 `tickets_evalues_fake.json`

AUTEUR: Widad

Base de tickets fictifs et leurs évaluations également fictives.

Variables retenues:

- `id_ticket`
- `contexte`
- `ticket`
- `modele`
- `dateheure`
- `reponse`
- `feedback_utilisateur`
- `temps_reponse (s)`

- `nombre_tokens`

Les améliorations à faire ici: il faut mettre à jour les noms de modèles si d'autres que ceux-ci sont utilisés + faire un programme qui génère automatiquement ce fichier à partir de vrais (tickets,réponses) + il faut faire le lien avec `evaluation_manager.py` qui récupère les feedbacks utilisateur (ajouter une option "moyennement utile" ici) + également il faut remplacer "ERREUR" par les vrais messages d'erreur qu'on obtient qu'on l'agent se plante (éventuellement définir un tel message si pas encore fait).

3 `config_evaluation.py`

AUTEUR: Widad

Ce fichier est en gros la formule qu'on utilise pour calculer le score qui va être attribué à chaque ticket évalué. Il peut être édité à volonté par exemple pour faire varier le poids attribué à chaque variable. Pour cette même raison c'est un fichier séparé du code d'évaluation.

Pour le moment on part sur 4 variables:

- `erreur_penalty` (si erreur, on pénalise avec un score de 0)
- `pertinence` (feedback utilisateur: utile, pas utile, moyennement utile)
- `vitesse` (temps de réponse)
- `efficacité` (nombre de tokens)

Ensuite on combine ces 4 critères avec des poids différents pour obtenir un score par ticket. J'ai fait le choix (modifiable) de prendre 35% pertinence, 25% pénalité erreur, 20% vitesse, et 20% efficacité. Ainsi la pertinence est le critère le plus important, les erreurs comptent, la vitesse et l'efficacité comptent un peu moins.

Pour calculer les `sous_scores` pour chaque critère (vitesse, pertinence etc) le feedback textuel est converti en valeurs numériques, et de même les valeurs temporelles (continues) relatives à la vitesse sont converties en scores suivant des seuils (par exemple: moins d' 1.0s = excellent, plus de 30.0 = lent, etc).

4 `tickets_evalues_scores.json`

AUTEUR: Widad

Ici on stocke les scores et sous-scores attribués aux tickets.